# FRUBBER POS

## Documento de Diseño de Base de Datos

Versión: 1.0

Estado: Aprobado

Fecha: 28 de septiembre de 2026

Responsable del producto: Jhoan Rivera

## Propósito del documento

Este documento define el diseño lógico y físico de la base de datos local de Frubber POS. Especifica tablas, columnas, tipos SQLite, claves, relaciones, restricciones, índices, vistas, disparadores, transacciones, migraciones y reglas de conservación histórica.

El diseño implementa la Visión y alcance 1.1, la SRS 1.1, las Reglas de negocio 1.0 y la Arquitectura técnica 1.0 aprobadas. No introduce funciones comerciales fuera de esos documentos.

## 1. Fuentes y alcance

1.1 Documentos normativos

• Documento de Visión y Alcance, versión 1.1 aprobada.

• Especificación de Requisitos de Software, versión 1.1 aprobada.

• Documento de Reglas de Negocio, versión 1.0 aprobada.

• Documento de Arquitectura Técnica, versión 1.0 aprobada.

1.2 Alcance persistente

La base almacenará usuarios, productos, existencia actual, cajas, ventas, líneas históricas, anulaciones, configuración del negocio, versión de esquema y estado técnico de prueba o licencia.

1.3 Exclusiones deliberadas

No se crearán tablas de clientes, categorías, subcategorías, precio de compra, impuestos, pagos combinados, egresos, devoluciones parciales, auditoría de ajustes de inventario, reportes, exportaciones ni sincronización entre equipos.

## 2. Tecnología y convenciones

2.1 Motor y acceso

Motor: SQLite embebido.

Proveedor: Microsoft.Data.Sqlite.

Acceso: Dapper con SQL explícito y parametrizado.

2.2 Parámetros de conexión

• PRAGMA foreign_keys = ON en cada conexión.

• PRAGMA journal_mode = WAL.

• PRAGMA synchronous = FULL.

• busy_timeout inicial de 5000 milisegundos.

• Transacciones explícitas para escrituras críticas.

2.3 Convenciones de nombres

Las tablas y columnas usarán nombres en inglés y PascalCase para coincidir con el modelo C#. Las claves primarias terminarán en Id y las claves foráneas conservarán el nombre de la entidad referenciada.

2.4 Tipos físicos

• INTEGER para claves, dinero, cantidades escaladas, booleanos, fechas y horas.

• TEXT para nombres, códigos, enumeraciones y sumas de verificación.

• BLOB para sal, hash y sobres protegidos mediante DPAPI.

• No se utilizará REAL para dinero ni cantidades.

2.5 Fechas y horas

Los hechos históricos usarán un instante UTC como entero Unix en milisegundos y conservarán el desplazamiento local original en minutos. La interfaz reconstruirá la fecha local histórica mediante ambos valores.

2.6 Dinero

Todos los importes se almacenarán como INTEGER en pesos colombianos, sin centavos.

2.7 Cantidades

Todas las cantidades y existencias se almacenarán como INTEGER en milésimas. Una unidad equivale a 1000; 1,250 kg equivale a 1250. Para UNIT solo se aceptarán múltiplos de 1000.

2.8 Booleanos

Los booleanos usarán INTEGER con CHECK que permita únicamente 0 o 1.

2.9 Política de eliminación

Las relaciones históricas usarán ON DELETE RESTRICT o NO ACTION. No se utilizará eliminación en cascada sobre ventas, líneas, cajas, anulaciones ni usuarios relacionados.

2.10 Tablas STRICT

Todas las tablas se crearán con la opción STRICT de SQLite. Los únicos tipos físicos serán INTEGER, TEXT y BLOB según este documento; cualquier valor de tipo incompatible se rechazará.

## 3. Modelo lógico

3.1 Entidades principales

• User: identidad operativa, rol, estado y material derivado del PIN.

• Product: catálogo, precio y existencia actual independiente.

• CashSession: apertura, cierre e instantánea final de totales.

• Sale: cabecera financiera e histórica.

• SaleLine: detalle histórico inmutable de productos vendidos.

• SaleVoid: evento de anulación completa.

• BusinessSetting: datos configurables del negocio e impresora.

3.2 Entidades técnicas

• SchemaMigration: scripts aplicados al esquema.

• SystemSequence: consecutivo global de ventas.

• LicenseState: sobre DPAPI de prueba y activación.

3.3 Relaciones

User 1—N CashSession como usuario de apertura o cierre.

User 1—N Sale como cajero.

User 1—N SaleVoid como administrador que anula.

CashSession 1—N Sale.

Sale 1—N SaleLine.

Sale 1—0..1 SaleVoid.

Product 1—N SaleLine.

### 3.4 Diagramas UML

*Figura 1. Núcleo comercial e integridad histórica.*

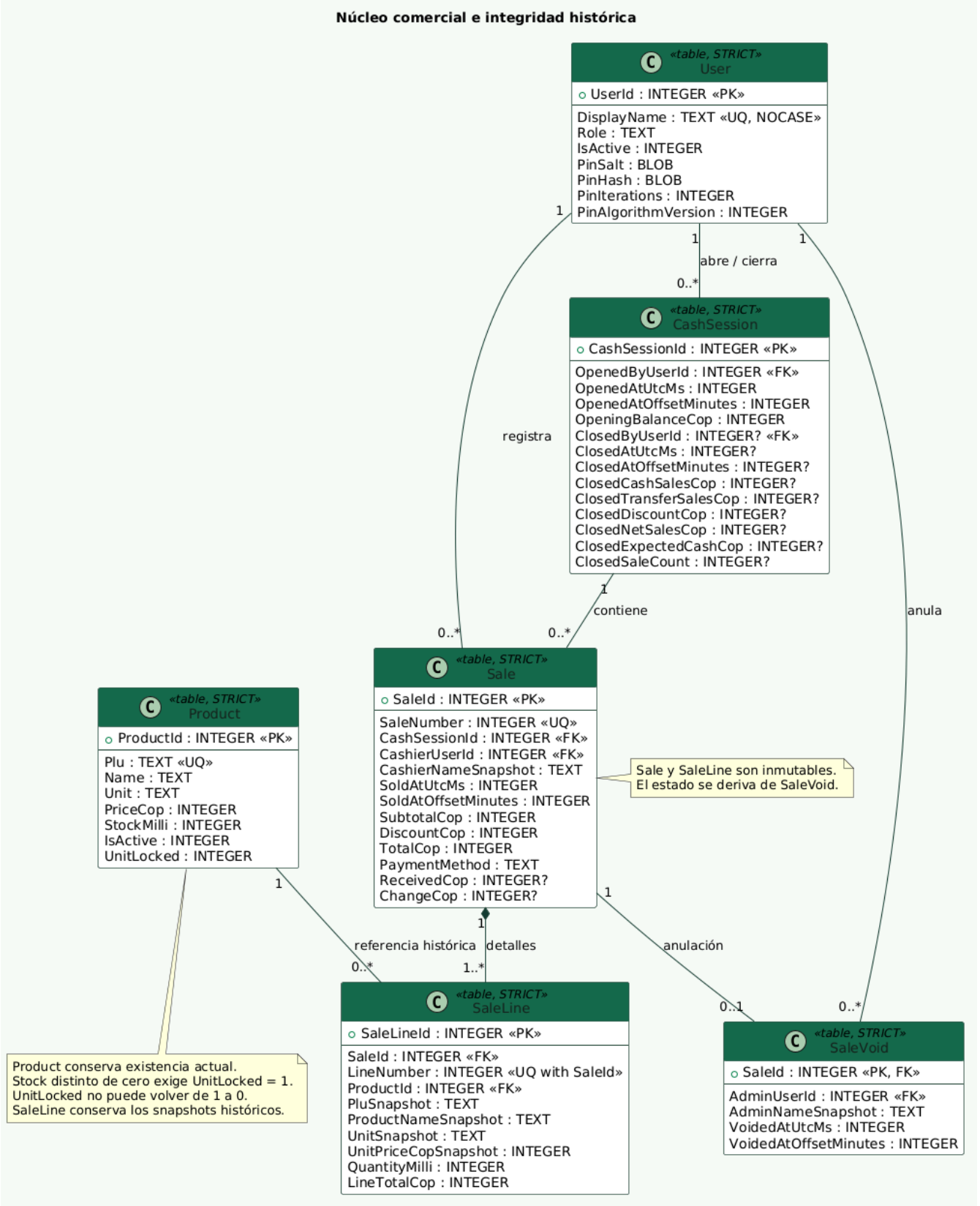
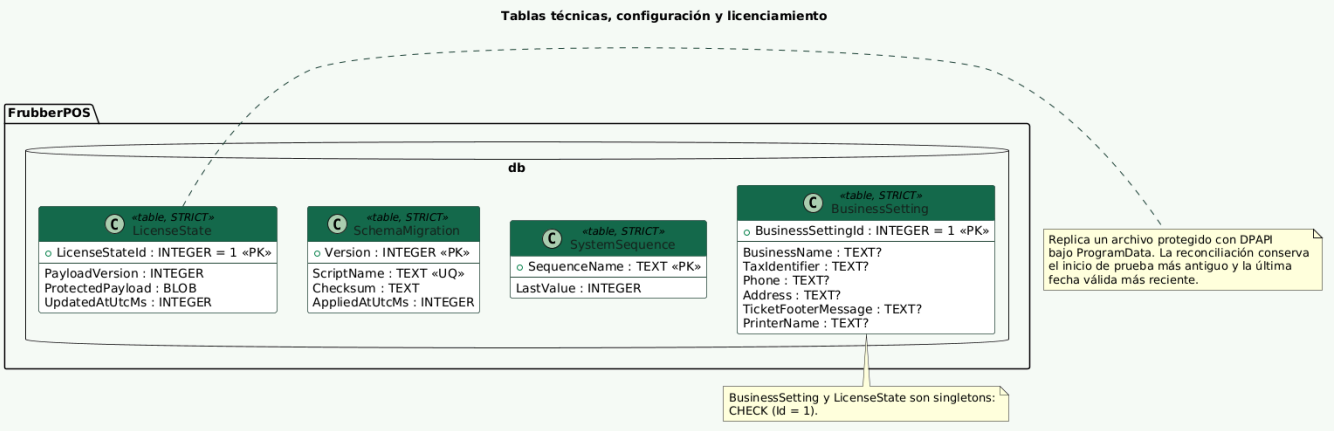



*Figura 2. Tablas técnicas, configuración y licenciamiento.*




Los diagramas utilizan clases con estereotipo <<table, STRICT>>, atributos con tipos SQLite, marcas PK/FK/UQ y multiplicidades UML. User.DisplayName identifica además su unicidad NOCASE. El diccionario de datos y el DDL mantienen la definición normativa de restricciones que no puede expresarse completamente en una imagen.

## 4. Diccionario de tablas

4.1 SchemaMigration

Propósito: registrar migraciones completadas.

Columnas:

• Version INTEGER, PK, mayor que cero.

• ScriptName TEXT, NOT NULL, UNIQUE.

• Checksum TEXT, NOT NULL.

• AppliedAtUtcMs INTEGER, NOT NULL.

Reglas: un script aplicado no puede cambiar de nombre, orden ni suma de verificación. Una versión futura desconocida bloqueará el inicio.

4.2 SystemSequence

Propósito: administrar consecutivos globales sin depender de presentación.

Columnas:

• SequenceName TEXT, PK.

• LastValue INTEGER, NOT NULL, CHECK LastValue >= 0.

Fila inicial: SequenceName = 'SaleNumber', LastValue = 0.

Regla: el incremento y la inserción de Sale ocurren dentro de la misma transacción. El formato V-000001 se genera en la aplicación.

4.3 User

Propósito: almacenar usuarios seleccionables para inicio de sesión.

Columnas:

• UserId INTEGER, PK.

• DisplayName TEXT, NOT NULL.

• Role TEXT, NOT NULL, CHECK Role IN ('ADMIN','CASHIER').

• IsActive INTEGER, NOT NULL, CHECK IsActive IN (0,1).

• PinSalt BLOB, NOT NULL, longitud 16 bytes.

• PinHash BLOB, NOT NULL, longitud 32 bytes.

• PinIterations INTEGER, NOT NULL, mayor que cero.

• PinAlgorithmVersion INTEGER, NOT NULL, mayor que cero.

Restricciones: DisplayName se guarda con Trim, no puede quedar vacío, admite máximo 60 caracteres y es único sin distinguir mayúsculas mediante UX_User_DisplayName_NoCase. El PIN nunca se almacena. Un usuario referenciado por ventas, cajas o anulaciones no puede eliminarse; solo desactivarse.

4.4 Product

Propósito: almacenar el catálogo y la existencia actual independiente.

Columnas:

• ProductId INTEGER, PK.

• Plu TEXT, NOT NULL.

• Name TEXT, NOT NULL.

• Unit TEXT, NOT NULL, CHECK Unit IN ('KG','LB','UNIT').

• PriceCop INTEGER, NOT NULL, CHECK PriceCop > 0.

• StockMilli INTEGER, NOT NULL.

• IsActive INTEGER, NOT NULL, CHECK IsActive IN (0,1).

• UnitLocked INTEGER, NOT NULL, CHECK UnitLocked IN (0,1).

Restricciones:

• Plu se guarda con Trim, en mayúsculas, y no puede quedar vacío.

• Plu admite únicamente caracteres alfanuméricos aprobados.

• La unicidad de Plu no distingue mayúsculas y minúsculas.

• Name se guarda con Trim y no puede quedar vacío.

• Cuando Unit = 'UNIT', StockMilli debe ser múltiplo de 1000.

• Cuando StockMilli sea distinto de cero, UnitLocked deberá ser 1.

• UnitLocked solo puede cambiar de 0 a 1; un trigger impide regresar de 1 a 0.

• Si UnitLocked = 1, un UPDATE no puede cambiar Unit.

• Product puede eliminarse solamente si no tiene SaleLine asociada.

Nota: no existe unicidad sobre Name. Cada Product administra una existencia independiente y la base no intenta detectar inventario físico compartido.

Regla aprobada: una existencia inicial distinta de cero activa UnitLocked desde la creación. Con existencia inicial cero, la unidad seguirá editable únicamente hasta la primera venta o ajuste.

4.5 CashSession

Propósito: representar la única caja operativa y conservar su cierre.

Columnas:

• CashSessionId INTEGER, PK.

• OpenedAtUtcMs INTEGER, NOT NULL.

• OpenedAtOffsetMinutes INTEGER, NOT NULL.

• OpenedByUserId INTEGER, FK User, NOT NULL.

• OpenedByNameSnapshot TEXT, NOT NULL.

• OpeningBalanceCop INTEGER, NOT NULL, CHECK OpeningBalanceCop >= 0.
• ClosedAtUtcMs INTEGER, NULL.
• ClosedAtOffsetMinutes INTEGER, NULL.
• ClosedByUserId INTEGER, FK User, NULL.
• ClosedByNameSnapshot TEXT, NULL.
• ClosedCashSalesCop INTEGER, NULL.
• ClosedTransferSalesCop INTEGER, NULL.
• ClosedDiscountCop INTEGER, NULL.
• ClosedNetSalesCop INTEGER, NULL.
• ClosedExpectedCashCop INTEGER, NULL.
• ClosedSaleCount INTEGER, NULL.
Restricciones:
• Una sesión está abierta cuando todos los campos de cierre son NULL.
• Una sesión está cerrada cuando todos los campos de cierre están completos.
• ClosedNetSalesCop = ClosedCashSalesCop + ClosedTransferSalesCop.
• ClosedExpectedCashCop = OpeningBalanceCop + ClosedCashSalesCop.
• Los totales y cantidades de cierre no pueden ser negativos.
• Un índice único parcial garantiza una sola CashSession abierta.
Regla aprobada: OpeningBalanceCop no puede ser negativo.

4.6 Sale
Propósito: cabecera histórica y financiera de una venta confirmada.
Columnas:
• SaleId INTEGER, PK interna.
• SaleNumber INTEGER, NOT NULL, UNIQUE, mayor que cero.
• CashSessionId INTEGER, FK CashSession, NOT NULL.
• CashierUserId INTEGER, FK User, NOT NULL.
• CashierNameSnapshot TEXT, NOT NULL.
• SoldAtUtcMs INTEGER, NOT NULL.
• SoldAtOffsetMinutes INTEGER, NOT NULL.
• SubtotalCop INTEGER, NOT NULL, mayor que cero.
• DiscountCop INTEGER, NOT NULL.
• TotalCop INTEGER, NOT NULL.
• PaymentMethod TEXT, NOT NULL, CHECK IN ('CASH','TRANSFER').
• ReceivedCop INTEGER, NULL.
• ChangeCop INTEGER, NULL.
Restricciones:
• DiscountCop entre cero y SubtotalCop.
• TotalCop = SubtotalCop - DiscountCop.
• Para CASH, ReceivedCop no es NULL, ReceivedCop >= TotalCop y ChangeCop = ReceivedCop - TotalCop.
• Para TRANSFER, ReceivedCop y ChangeCop son NULL.

• Sale no contiene estado mutable. El estado se deriva de SaleVoid.
• Sale no se actualiza ni elimina después del commit.

4.7 SaleLine
Propósito: conservar la fotografía histórica de cada producto vendido.
Columnas:
• SaleLineId INTEGER, PK.
• SaleId INTEGER, FK Sale, NOT NULL.
• LineNumber INTEGER, NOT NULL, mayor que cero.
• ProductId INTEGER, FK Product, NOT NULL.
• PluSnapshot TEXT, NOT NULL.
• ProductNameSnapshot TEXT, NOT NULL.
• UnitSnapshot TEXT, NOT NULL, CHECK IN ('KG','LB','UNIT').
• UnitPriceCopSnapshot INTEGER, NOT NULL, mayor que cero.
• QuantityMilli INTEGER, NOT NULL, mayor que cero.
• LineTotalCop INTEGER, NOT NULL, mayor o igual a cero.
Restricciones:
• UNIQUE (SaleId, LineNumber).
• Para UnitSnapshot = 'UNIT', QuantityMilli debe ser múltiplo de 1000.
• LineTotalCop = (UnitPriceCopSnapshot × QuantityMilli + 500) / 1000 usando aritmética entera positiva.
• Los campos Snapshot son obligatorios y no se reconstruyen desde Product.
• SaleLine no se actualiza ni elimina después del commit.

4.8 SaleVoid
Propósito: registrar una anulación completa sin modificar la venta original.
Columnas:
• SaleId INTEGER, PK y FK Sale.
• VoidedAtUtcMs INTEGER, NOT NULL.
• VoidedAtOffsetMinutes INTEGER, NOT NULL.
• AdminUserId INTEGER, FK User, NOT NULL.
• AdminNameSnapshot TEXT, NOT NULL.
Restricciones:
• La PK impide anular dos veces.
• No se almacena motivo.
• El usuario debe ser ADMIN activo.
• La CashSession de la venta debe continuar abierta.
• SaleVoid no se actualiza ni elimina.

4.9 BusinessSetting
Propósito: almacenar configuración básica del negocio y de impresión.
Columnas y longitudes máximas aprobadas:

• BusinessSettingId INTEGER, PK, CHECK BusinessSettingId = 1.

• BusinessName TEXT, NULL, máximo 120.

• TaxIdentifier TEXT, NULL, máximo 32.

• Phone TEXT, NULL, máximo 30.

• Address TEXT, NULL, máximo 200.

• TicketFooterMessage TEXT, NULL, máximo 250.

• PrinterName TEXT, NULL, máximo 260.

Reglas: existirá como máximo una fila. Los valores se guardan con Trim y una cadena vacía se convierte en NULL. PrinterName identifica una impresora reconocida por Windows; el ancho efectivo proviene del controlador y no se almacena como regla fija.

4.10 LicenseState

Propósito: conservar una copia redundante del estado técnico protegido.

Columnas:

• LicenseStateId INTEGER, PK, CHECK LicenseStateId = 1.

• PayloadVersion INTEGER, NOT NULL, mayor que cero.

• ProtectedPayload BLOB, NOT NULL.

• UpdatedAtUtcMs INTEGER, NOT NULL.

Reglas: ProtectedPayload es un sobre DPAPI LocalMachine. Su contenido lógico incluye huella, inicio de prueba, última fecha válida, activación e identificador de licencia. Se reconcilia con la copia de ProgramData.

## 5. Restricciones e integridad

5.1 Integridad referencial

Todas las claves foráneas históricas usan RESTRICT o NO ACTION. No habrá ON DELETE CASCADE sobre entidades comerciales.

5.2 Inmutabilidad histórica

Disparadores rechazarán UPDATE y DELETE sobre Sale, SaleLine y SaleVoid durante la operación normal.

5.3 Producto y unidad

Un disparador impedirá cambiar Product.Unit cuando UnitLocked = 1 y otro impedirá que UnitLocked regrese de 1 a 0. La aplicación activará UnitLocked al registrar la primera venta o ajuste.

5.4 Caja única

Un índice único parcial sobre una expresión constante WHERE ClosedAtUtcMs IS NULL impedirá más de una caja abierta.

5.5 Anulación

Un disparador previo a SaleVoid comprobará caja abierta y rol ADMIN. El reintegro del inventario se hará explícitamente dentro de la misma transacción de aplicación.

5.6 Totales

La aplicación recalculará líneas, subtotal, descuento, total y pago antes del commit. Las restricciones CHECK protegerán relaciones aritméticas dentro de cada fila.

## 6. Índices

6.1 User

• UX_User_DisplayName_NoCase, único, sobre DisplayName COLLATE NOCASE.

• IX_User_Active_DisplayName sobre IsActive y DisplayName COLLATE NOCASE.

6.2 Product

• UX_Product_Plu_NoCase, único, sobre Plu COLLATE NOCASE.

• IX_Product_Active_Name sobre IsActive y Name COLLATE NOCASE.

6.3 CashSession

• UX_CashSession_OnlyOneOpen, único y parcial, WHERE ClosedAtUtcMs IS NULL.

• IX_CashSession_OpenedAt sobre OpenedAtUtcMs.

6.4 Sale

• UX_Sale_SaleNumber, único.

• IX_Sale_SoldAt sobre SoldAtUtcMs.

• IX_Sale_CashSession_SoldAt sobre CashSessionId y SoldAtUtcMs.

• IX_Sale_Cashier_SoldAt sobre CashierUserId y SoldAtUtcMs.

6.5 SaleLine

• UX_SaleLine_Sale_LineNumber, único.

• IX_SaleLine_Product sobre ProductId.

6.6 SaleVoid

SaleId es índice único por ser PK. Se agregará IX_SaleVoid_VoidedAt sobre VoidedAtUtcMs si las pruebas de historial lo justifican.

## 7. Vistas de consulta

7.1 V_SaleHistory

Derivará SaleStatus como ACTIVE o VOIDED mediante LEFT JOIN con SaleVoid. Expondrá cabecera, cajero, caja, totales y datos de anulación sin consultar valores actuales de Product.

7.2 V_OpenCashTotals

Sumará únicamente ventas sin SaleVoid pertenecientes a la caja abierta. Calculará efectivo, transferencia, descuentos, venta neta, cantidad de ventas y efectivo esperado.

7.3 Cierre histórico

Las cajas cerradas no dependerán de vistas recalculadas: se mostrarán los campos Closed de CashSession.

## 8. Disparadores previstos

• TR_Sale_NoUpdate.

• TR_Sale_NoDelete.

• TR_SaleLine_NoUpdate.

• TR_SaleLine_NoDelete.

• TR_SaleVoid_NoUpdate.

• TR_SaleVoid_NoDelete.

• TR_Product_BlockUnitChange.

• TR_Product_PreventUnitUnlock.

• TR_SaleVoid_RequireOpenCashAndAdmin.

Los scripts exactos forman parte de la migración 001_Initial.sql y se validaron ejecutándolos sobre una base SQLite vacía. La pestaña 001_Initial.sql de este documento contiene el DDL normativo completo.

## 9. Transacciones

9.1 Registro de venta

Una única transacción deberá:

1. Confirmar que existe una caja abierta.

2. Validar usuario activo y autorizado.

3. Obtener e incrementar SaleNumber.

4. Insertar Sale.

5. Insertar una o más SaleLine con snapshots.

6. Descontar Product.StockMilli y activar UnitLocked.

7. Recalcular y verificar subtotal y total.

8. Confirmar o revertir todo.

9.2 Anulación

Una única transacción deberá:

1. Validar ADMIN activo.

2. Confirmar ausencia de SaleVoid.

3. Confirmar que la caja asociada sigue abierta.

4. Insertar SaleVoid.

5. Reintegrar las cantidades de todas las líneas.

6. Confirmar o revertir todo.

9.3 Apertura

La inserción de CashSession dependerá del índice de caja única. OpeningBalanceCop se almacena como entero COP.

9.4 Cierre

Una única transacción calculará ventas vigentes, escribirá todos los campos Closed y dejará la sesión inmutable para fines funcionales.

9.5 Ajuste de inventario

Un UPDATE directo reemplazará StockMilli y activará UnitLocked. No se insertará registro de auditoría ni motivo.

## 10. Migraciones

10.1 Migración inicial

El esquema inicial se entrega como 001_Initial.sql embebido en Infrastructure. Crea diez tablas STRICT, índices, dos vistas, doce triggers y las filas iniciales de SystemSequence y BusinessSetting. La pestaña 001_Initial.sql de este documento contiene su texto normativo completo. SHA-256 validado: BA92B0F532CCEFCDE76234B66AF06BF8C7F21EF9E01EF2820E6CA8426DCE400A.

10.2 Registro

SchemaMigration conservará versión, nombre, checksum y fecha de aplicación.

10.3 Ejecución

Antes de migrar se creará un respaldo consistente. Cada script se ejecutará dentro de una transacción. Solo se registrará al finalizar correctamente.

10.4 Fallos

Una migración fallida revertirá su transacción, bloqueará el inicio normal y conservará respaldo y log. Una aplicación antigua rechazará una base con versión futura.

## 11. Respaldo y restauración

El respaldo manual usará SQLite Backup API hacia un temporal, ejecutará una comprobación de integridad y publicará únicamente una copia válida. La restauración no se ofrecerá desde la interfaz y se realizará mediante procedimiento técnico externo.

## 12. Volumen y rendimiento

El diseño deberá soportar al menos 5.000 productos y 100.000 ventas. Las consultas de catálogo usarán índices de PLU y nombre; el historial usará SoldAtUtcMs; los totales de caja usarán CashSessionId y la ausencia de SaleVoid. Los planes de consulta se verificarán con datos de volumen representativo.

## 13. Trazabilidad resumida

• Productos e inventario: RF-PRO, RF-UNI, RF-INV; RN-PRO, RN-UNI, RN-INV.

• Caja: RF-CAJ; RN-CAJ.

• Ventas y pagos: RF-VEN; RN-VEN.

• Historial y anulaciones: RF-HIS, RF-ANU; RN-HIS.

• Usuarios: RF-AUT; RN-USR.

• Configuración e impresión: RF-IMP y sección de configuración del negocio.

• Datos: DAT-001 a DAT-016.

• Licencia y respaldo: RN-LIC, RN-BKP y ADR-012 a ADR-015.

## 14. Decisiones aprobadas

**P-DB-001 — Aprobada. Límites: PLU 32; producto 120; usuario 60; negocio 120; identificación 32; teléfono 30; dirección 200; mensaje final 250; impresora 260.**

**P-DB-002 — Aprobada. DisplayName es único mediante comparación NOCASE.**

**P-DB-003 — Aprobada. Una existencia inicial distinta de cero activa UnitLocked.**

**P-DB-004 — Aprobada. OpeningBalanceCop debe ser mayor o igual a cero.**

**P-DB-005 — Aprobada. Todas las tablas utilizan STRICT.**

### 14.1 Resultado de validación

**La migración se ejecutó correctamente sobre SQLite 3.46.1 en una base vacía. Se comprobaron la unicidad de usuario, bloqueo irreversible de unidad con existencia, caja abierta única, registro y anulación de venta, redondeo, inmutabilidad, totales de caja e integrity_check. El esquema resultante contiene diez tablas, dos vistas y doce triggers.**

## 15. Aprobación

Versión aprobada: 1.0

Estado de aprobación: Aprobado

Responsable de aprobación: Jhoan Rivera

Fecha de aprobación: 28 de septiembre de 2026

Fin del documento

```sql
-- Frubber POS
-- Migración: 001_Initial.sql
-- Diseño de base de datos: versión aprobada 1.0
-- El ejecutor de migraciones abre la transacción. Este script no ejecuta BEGIN/COMMIT.
-- La conexión debe habilitar foreign_keys, WAL, synchronous FULL y busy_timeout.

CREATE TABLE SchemaMigration (
    Version INTEGER NOT NULL PRIMARY KEY CHECK (Version > 0),
    ScriptName TEXT NOT NULL UNIQUE
        CHECK (ScriptName = trim(ScriptName) AND length(ScriptName) BETWEEN 1 AND 120),
    Checksum TEXT NOT NULL
        CHECK (
            length(Checksum) = 64
            AND Checksum = upper(Checksum)
            AND Checksum NOT GLOB '*[^0-9A-F]*'
        ),
    AppliedAtUtcMs INTEGER NOT NULL CHECK (AppliedAtUtcMs > 0)
) STRICT;

CREATE TABLE SystemSequence (
    SequenceName TEXT NOT NULL PRIMARY KEY
        CHECK (SequenceName = trim(SequenceName) AND length(SequenceName) BETWEEN 1 AND 60),
    LastValue INTEGER NOT NULL CHECK (LastValue >= 0)
) STRICT;

CREATE TABLE "User" (
    UserId INTEGER PRIMARY KEY,
    DisplayName TEXT NOT NULL
        CHECK (
            DisplayName = trim(DisplayName)
            AND length(DisplayName) BETWEEN 1 AND 60
        ),
    Role TEXT NOT NULL CHECK (Role IN ('ADMIN', 'CASHIER')),
    IsActive INTEGER NOT NULL CHECK (IsActive IN (0, 1)),
    PinSalt BLOB NOT NULL CHECK (length(PinSalt) = 16),
    PinHash BLOB NOT NULL CHECK (length(PinHash) = 32),
    PinIterations INTEGER NOT NULL CHECK (PinIterations > 0),
    PinAlgorithmVersion INTEGER NOT NULL CHECK (PinAlgorithmVersion > 0)
) STRICT;

CREATE TABLE Product (
    ProductId INTEGER PRIMARY KEY,
    Plu TEXT NOT NULL
        CHECK (
            Plu = trim(Plu)
            AND Plu = upper(Plu)
            AND length(Plu) BETWEEN 1 AND 32
            AND Plu NOT GLOB '*[^A-Z0-9]*'
        ),
    Name TEXT NOT NULL
        CHECK (
            Name = trim(Name)
            AND length(Name) BETWEEN 1 AND 120
        ),
    Unit TEXT NOT NULL CHECK (Unit IN ('KG', 'LB', 'UNIT')),
    PriceCop INTEGER NOT NULL CHECK (PriceCop > 0),
    StockMilli INTEGER NOT NULL,
    IsActive INTEGER NOT NULL CHECK (IsActive IN (0, 1)),
    UnitLocked INTEGER NOT NULL CHECK (UnitLocked IN (0, 1)),
    CHECK (Unit <> 'UNIT' OR StockMilli % 1000 = 0),
    CHECK (StockMilli = 0 OR UnitLocked = 1)
) STRICT;

CREATE TABLE CashSession (
    CashSessionId INTEGER PRIMARY KEY,
    OpenedAtUtcMs INTEGER NOT NULL CHECK (OpenedAtUtcMs > 0),
    OpenedAtOffsetMinutes INTEGER NOT NULL CHECK (OpenedAtOffsetMinutes BETWEEN -840 AND 840),
    OpenedByUserId INTEGER NOT NULL,
    OpenedByNameSnapshot TEXT NOT NULL
        CHECK (
            OpenedByNameSnapshot = trim(OpenedByNameSnapshot)
            AND length(OpenedByNameSnapshot) BETWEEN 1 AND 60
        ),
    OpeningBalanceCop INTEGER NOT NULL CHECK (OpeningBalanceCop >= 0),
```

```
    ClosedAtUtcMs INTEGER,
    ClosedAtOffsetMinutes INTEGER,
    ClosedByUserId INTEGER,
    ClosedByNameSnapshot TEXT,
    ClosedCashSalesCop INTEGER,
    ClosedTransferSalesCop INTEGER,
    ClosedDiscountCop INTEGER,
    ClosedNetSalesCop INTEGER,
    ClosedExpectedCashCop INTEGER,
    ClosedSaleCount INTEGER,
    FOREIGN KEY (OpenedByUserId) REFERENCES "User" (UserId) ON DELETE RESTRICT,
    FOREIGN KEY (ClosedByUserId) REFERENCES "User" (UserId) ON DELETE RESTRICT,
    CHECK (
        (
            ClosedAtUtcMs IS NULL
            AND ClosedAtOffsetMinutes IS NULL
            AND ClosedByUserId IS NULL
            AND ClosedByNameSnapshot IS NULL
            AND ClosedCashSalesCop IS NULL
            AND ClosedTransferSalesCop IS NULL
            AND ClosedDiscountCop IS NULL
            AND ClosedNetSalesCop IS NULL
            AND ClosedExpectedCashCop IS NULL
            AND ClosedSaleCount IS NULL
        )
        OR
        (
            ClosedAtUtcMs IS NOT NULL
            AND ClosedAtUtcMs > 0
            AND ClosedAtOffsetMinutes IS NOT NULL
            AND ClosedAtOffsetMinutes BETWEEN -840 AND 840
            AND ClosedByUserId IS NOT NULL
            AND ClosedByNameSnapshot IS NOT NULL
            AND ClosedByNameSnapshot = trim(ClosedByNameSnapshot)
            AND length(ClosedByNameSnapshot) BETWEEN 1 AND 60
            AND ClosedCashSalesCop IS NOT NULL
            AND ClosedCashSalesCop >= 0
            AND ClosedTransferSalesCop IS NOT NULL
            AND ClosedTransferSalesCop >= 0
            AND ClosedDiscountCop IS NOT NULL
            AND ClosedDiscountCop >= 0
            AND ClosedNetSalesCop IS NOT NULL
            AND ClosedNetSalesCop >= 0
            AND ClosedExpectedCashCop IS NOT NULL
            AND ClosedExpectedCashCop >= 0
            AND ClosedSaleCount IS NOT NULL
            AND ClosedSaleCount >= 0
            AND ClosedNetSalesCop = ClosedCashSalesCop + ClosedTransferSalesCop
            AND ClosedExpectedCashCop = OpeningBalanceCop + ClosedCashSalesCop
        )
    )
) STRICT;

CREATE TABLE Sale (
    SaleId INTEGER PRIMARY KEY,
    SaleNumber INTEGER NOT NULL CHECK (SaleNumber > 0),
    CashSessionId INTEGER NOT NULL,
    CashierUserId INTEGER NOT NULL,
    CashierNameSnapshot TEXT NOT NULL
        CHECK (
            CashierNameSnapshot = trim(CashierNameSnapshot)
            AND length(CashierNameSnapshot) BETWEEN 1 AND 60
        ),
    SoldAtUtcMs INTEGER NOT NULL CHECK (SoldAtUtcMs > 0),
    SoldAtOffsetMinutes INTEGER NOT NULL CHECK (SoldAtOffsetMinutes BETWEEN -840 AND 840),
    SubtotalCop INTEGER NOT NULL CHECK (SubtotalCop > 0),
    DiscountCop INTEGER NOT NULL CHECK (DiscountCop >= 0 AND DiscountCop <= SubtotalCop),
    TotalCop INTEGER NOT NULL CHECK (TotalCop = SubtotalCop - DiscountCop),
    PaymentMethod TEXT NOT NULL CHECK (PaymentMethod IN ('CASH', 'TRANSFER')),
    ReceivedCop INTEGER,
    ChangeCop INTEGER,
    FOREIGN KEY (CashSessionId) REFERENCES CashSession (CashSessionId) ON DELETE RESTRICT,
    FOREIGN KEY (CashierUserId) REFERENCES "User" (UserId) ON DELETE RESTRICT,
    CHECK (
```

```
        (
            PaymentMethod = 'CASH'
            AND ReceivedCop IS NOT NULL
            AND ReceivedCop >= TotalCop
            AND ChangeCop IS NOT NULL
            AND ChangeCop = ReceivedCop - TotalCop
        )
        OR
        (
            PaymentMethod = 'TRANSFER'
            AND ReceivedCop IS NULL
            AND ChangeCop IS NULL
        )
    )
) STRICT;

CREATE TABLE SaleLine (
    SaleLineId INTEGER PRIMARY KEY,
    SaleId INTEGER NOT NULL,
    LineNumber INTEGER NOT NULL CHECK (LineNumber > 0),
    ProductId INTEGER NOT NULL,
    PluSnapshot TEXT NOT NULL
        CHECK (
            PluSnapshot = trim(PluSnapshot)
            AND length(PluSnapshot) BETWEEN 1 AND 32
        ),
    ProductNameSnapshot TEXT NOT NULL
        CHECK (
            ProductNameSnapshot = trim(ProductNameSnapshot)
            AND length(ProductNameSnapshot) BETWEEN 1 AND 120
        ),
    UnitSnapshot TEXT NOT NULL CHECK (UnitSnapshot IN ('KG', 'LB', 'UNIT')),
    UnitPriceCopSnapshot INTEGER NOT NULL CHECK (UnitPriceCopSnapshot > 0),
    QuantityMilli INTEGER NOT NULL CHECK (QuantityMilli > 0),
    LineTotalCop INTEGER NOT NULL CHECK (LineTotalCop >= 0),
    FOREIGN KEY (SaleId) REFERENCES Sale (SaleId) ON DELETE RESTRICT,
    FOREIGN KEY (ProductId) REFERENCES Product (ProductId) ON DELETE RESTRICT,
    CHECK (UnitSnapshot <> 'UNIT' OR QuantityMilli % 1000 = 0),
    CHECK (LineTotalCop = (UnitPriceCopSnapshot * QuantityMilli + 500) / 1000)
) STRICT;

CREATE TABLE SaleVoid (
    SaleId INTEGER PRIMARY KEY,
    VoidedAtUtcMs INTEGER NOT NULL CHECK (VoidedAtUtcMs > 0),
    VoidedAtOffsetMinutes INTEGER NOT NULL CHECK (VoidedAtOffsetMinutes BETWEEN -840 AND 840),
    AdminUserId INTEGER NOT NULL,
    AdminNameSnapshot TEXT NOT NULL
        CHECK (
            AdminNameSnapshot = trim(AdminNameSnapshot)
            AND length(AdminNameSnapshot) BETWEEN 1 AND 60
        ),
    FOREIGN KEY (SaleId) REFERENCES Sale (SaleId) ON DELETE RESTRICT,
    FOREIGN KEY (AdminUserId) REFERENCES "User" (UserId) ON DELETE RESTRICT
) STRICT;

CREATE TABLE BusinessSetting (
    BusinessSettingId INTEGER PRIMARY KEY CHECK (BusinessSettingId = 1),
    BusinessName TEXT
        CHECK (
            BusinessName IS NULL
            OR (BusinessName = trim(BusinessName) AND length(BusinessName) BETWEEN 1 AND 120)
        ),
    TaxIdentifier TEXT
        CHECK (
            TaxIdentifier IS NULL
            OR (TaxIdentifier = trim(TaxIdentifier) AND length(TaxIdentifier) BETWEEN 1 AND 32)
        ),
    Phone TEXT
        CHECK (
            Phone IS NULL
            OR (Phone = trim(Phone) AND length(Phone) BETWEEN 1 AND 30)
        ),
    Address TEXT
        CHECK (
```

```
            Address IS NULL
            OR (Address = trim(Address) AND length(Address) BETWEEN 1 AND 200)
        ),
    TicketFooterMessage TEXT
        CHECK (
            TicketFooterMessage IS NULL
            OR (TicketFooterMessage = trim(TicketFooterMessage) AND length(TicketFooterMessage) BETWEEN
1 AND 250)
        ),
    PrinterName TEXT
        CHECK (
            PrinterName IS NULL
            OR (PrinterName = trim(PrinterName) AND length(PrinterName) BETWEEN 1 AND 260)
        )
) STRICT;

CREATE TABLE LicenseState (
    LicenseStateId INTEGER PRIMARY KEY CHECK (LicenseStateId = 1),
    PayloadVersion INTEGER NOT NULL CHECK (PayloadVersion > 0),
    ProtectedPayload BLOB NOT NULL CHECK (length(ProtectedPayload) > 0),
    UpdatedAtUtcMs INTEGER NOT NULL CHECK (UpdatedAtUtcMs > 0)
) STRICT;

CREATE UNIQUE INDEX UX_User_DisplayName_NoCase
    ON "User" (DisplayName COLLATE NOCASE);

CREATE INDEX IX_User_Active_DisplayName
    ON "User" (IsActive, DisplayName COLLATE NOCASE);

CREATE UNIQUE INDEX UX_Product_Plu_NoCase
    ON Product (Plu COLLATE NOCASE);

CREATE INDEX IX_Product_Active_Name
    ON Product (IsActive, Name COLLATE NOCASE);

CREATE UNIQUE INDEX UX_CashSession_OnlyOneOpen
    ON CashSession ((1))
    WHERE ClosedAtUtcMs IS NULL;

CREATE INDEX IX_CashSession_OpenedAt
    ON CashSession (OpenedAtUtcMs);

CREATE UNIQUE INDEX UX_Sale_SaleNumber
    ON Sale (SaleNumber);

CREATE INDEX IX_Sale_SoldAt
    ON Sale (SoldAtUtcMs);

CREATE INDEX IX_Sale_CashSession_SoldAt
    ON Sale (CashSessionId, SoldAtUtcMs);

CREATE INDEX IX_Sale_Cashier_SoldAt
    ON Sale (CashierUserId, SoldAtUtcMs);

CREATE UNIQUE INDEX UX_SaleLine_Sale_LineNumber
    ON SaleLine (SaleId, LineNumber);

CREATE INDEX IX_SaleLine_Product
    ON SaleLine (ProductId);

CREATE INDEX IX_SaleVoid_VoidedAt
    ON SaleVoid (VoidedAtUtcMs);

CREATE VIEW V_SaleHistory AS
SELECT
    s.SaleId,
    s.SaleNumber,
    s.CashSessionId,
    s.CashierUserId,
    s.CashierNameSnapshot,
    s.SoldAtUtcMs,
    s.SoldAtOffsetMinutes,
    s.SubtotalCop,
    s.DiscountCop,
```

```
    s.TotalCop,
    s.PaymentMethod,
    s.ReceivedCop,
    s.ChangeCop,
    CASE WHEN sv.SaleId IS NULL THEN 'ACTIVE' ELSE 'VOIDED' END AS SaleStatus,
    sv.VoidedAtUtcMs,
    sv.VoidedAtOffsetMinutes,
    sv.AdminUserId,
    sv.AdminNameSnapshot
FROM Sale AS s
LEFT JOIN SaleVoid AS sv ON sv.SaleId = s.SaleId;

CREATE VIEW V_OpenCashTotals AS
SELECT
    cs.CashSessionId,
    cs.OpeningBalanceCop,
    COALESCE(SUM(CASE
        WHEN sv.SaleId IS NULL AND s.PaymentMethod = 'CASH' THEN s.TotalCop
        ELSE 0
    END), 0) AS CashSalesCop,
    COALESCE(SUM(CASE
        WHEN sv.SaleId IS NULL AND s.PaymentMethod = 'TRANSFER' THEN s.TotalCop
        ELSE 0
    END), 0) AS TransferSalesCop,
    COALESCE(SUM(CASE
        WHEN sv.SaleId IS NULL THEN s.DiscountCop
        ELSE 0
    END), 0) AS DiscountCop,
    COALESCE(SUM(CASE
        WHEN sv.SaleId IS NULL THEN s.TotalCop
        ELSE 0
    END), 0) AS NetSalesCop,
    COALESCE(SUM(CASE
        WHEN s.SaleId IS NOT NULL AND sv.SaleId IS NULL THEN 1
        ELSE 0
    END), 0) AS SaleCount,
    cs.OpeningBalanceCop + COALESCE(SUM(CASE
        WHEN sv.SaleId IS NULL AND s.PaymentMethod = 'CASH' THEN s.TotalCop
        ELSE 0
    END), 0) AS ExpectedCashCop
FROM CashSession AS cs
LEFT JOIN Sale AS s ON s.CashSessionId = cs.CashSessionId
LEFT JOIN SaleVoid AS sv ON sv.SaleId = s.SaleId
WHERE cs.ClosedAtUtcMs IS NULL
GROUP BY cs.CashSessionId, cs.OpeningBalanceCop;

CREATE TRIGGER TR_Product_BlockUnitChange
BEFORE UPDATE OF Unit ON Product
FOR EACH ROW
WHEN OLD.UnitLocked = 1 AND NEW.Unit <> OLD.Unit
BEGIN
    SELECT RAISE(ABORT, 'La unidad del producto está bloqueada por historial o existencia');
END;

CREATE TRIGGER TR_Product_PreventUnitUnlock
BEFORE UPDATE OF UnitLocked ON Product
FOR EACH ROW
WHEN OLD.UnitLocked = 1 AND NEW.UnitLocked = 0
BEGIN
    SELECT RAISE(ABORT, 'El bloqueo de unidad es irreversible');
END;

CREATE TRIGGER TR_CashSession_ProtectOpeningFields
BEFORE UPDATE ON CashSession
FOR EACH ROW
WHEN
    NEW.OpenedAtUtcMs IS NOT OLD.OpenedAtUtcMs
    OR NEW.OpenedAtOffsetMinutes IS NOT OLD.OpenedAtOffsetMinutes
    OR NEW.OpenedByUserId IS NOT OLD.OpenedByUserId
    OR NEW.OpenedByNameSnapshot IS NOT OLD.OpenedByNameSnapshot
    OR NEW.OpeningBalanceCop IS NOT OLD.OpeningBalanceCop
BEGIN
    SELECT RAISE(ABORT, 'Los datos de apertura de caja son inmutables');
END;
```

```
CREATE TRIGGER TR_CashSession_NoUpdateAfterClose
BEFORE UPDATE ON CashSession
FOR EACH ROW
WHEN OLD.ClosedAtUtcMs IS NOT NULL
BEGIN
    SELECT RAISE(ABORT, 'Una caja cerrada es inmutable');
END;

CREATE TRIGGER TR_Sale_RequireOpenCashAndActiveUser
BEFORE INSERT ON Sale
FOR EACH ROW
BEGIN
    SELECT CASE WHEN NOT EXISTS (
        SELECT 1
        FROM CashSession AS cs
        WHERE cs.CashSessionId = NEW.CashSessionId
          AND cs.ClosedAtUtcMs IS NULL
    ) THEN RAISE(ABORT, 'La venta requiere una caja abierta') END;

    SELECT CASE WHEN NOT EXISTS (
        SELECT 1
        FROM "User" AS u
        WHERE u.UserId = NEW.CashierUserId
          AND u.IsActive = 1
          AND u.Role IN ('ADMIN', 'CASHIER')
    ) THEN RAISE(ABORT, 'El usuario de la venta no está activo o autorizado') END;
END;

CREATE TRIGGER TR_Sale_NoUpdate
BEFORE UPDATE ON Sale
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Sale es inmutable');
END;

CREATE TRIGGER TR_Sale_NoDelete
BEFORE DELETE ON Sale
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Sale no puede eliminarse');
END;

CREATE TRIGGER TR_SaleLine_NoUpdate
BEFORE UPDATE ON SaleLine
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'SaleLine es inmutable');
END;

CREATE TRIGGER TR_SaleLine_NoDelete
BEFORE DELETE ON SaleLine
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'SaleLine no puede eliminarse');
END;

CREATE TRIGGER TR_SaleVoid_RequireOpenCashAndAdmin
BEFORE INSERT ON SaleVoid
FOR EACH ROW
BEGIN
    SELECT CASE WHEN NOT EXISTS (
        SELECT 1
        FROM Sale AS s
        JOIN CashSession AS cs ON cs.CashSessionId = s.CashSessionId
        WHERE s.SaleId = NEW.SaleId
          AND cs.ClosedAtUtcMs IS NULL
    ) THEN RAISE(ABORT, 'Solo puede anularse una venta de una caja abierta') END;

    SELECT CASE WHEN NOT EXISTS (
        SELECT 1
        FROM "User" AS u
        WHERE u.UserId = NEW.AdminUserId
          AND u.IsActive = 1
```

```
          AND u.Role = 'ADMIN'
    ) THEN RAISE(ABORT, 'La anulación requiere un administrador activo') END;
END;

CREATE TRIGGER TR_SaleVoid_NoUpdate
BEFORE UPDATE ON SaleVoid
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'SaleVoid es inmutable');
END;

CREATE TRIGGER TR_SaleVoid_NoDelete
BEFORE DELETE ON SaleVoid
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'SaleVoid no puede eliminarse');
END;

INSERT INTO SystemSequence (SequenceName, LastValue)
VALUES ('SaleNumber', 0);

INSERT INTO BusinessSetting (BusinessSettingId)
VALUES (1);
```